

파이토치 학습 기법 및 핵심 개념 요약

개념명	설명	관련 수식 또는 코드	주요 특징	비고 (추론)
모델 (Model)	알 수 없는 파라미터를 가진 함수로, 데이터에 맞춰가는(fitting) 과정의 핵심 구성 요소입니다.	$t_c = w \times t_u + b$	입출력 쌍을 활용하여 스스로 최적화하며 특정 작업에 학습된 일반화된 모델을 의미합니다.	케플러가 관찰 데이터를 바탕으로 행성 궤도를 타원으로 정의한 것처럼, 딥러닝에서도 데이터를 가장 잘 설명하는 함수 형태를 찾는 것이 중요합니다.
손실 함수 (Loss function)	모델의 출력값과 실측 자료 (ground truth) 사이의 오차를 측정하여 학습의 방향을 결정하는 함수입니다.	$SquaredDiffs = (t_p - t_c)^2$	훈련 샘플로부터 기대하는 값과 실제 출력값 사이의 차이를 계산하며, 완벽하게 일치할 경우 최소가 됩니다.	오차가 높을수록 출력값이 커지도록 정의하며, 파라미터 조정은 손실값이 큰 샘플을 우선적으로 보정하도록 동작합니다.
경사 하강법 (Gradient Descent)	각 파라미터와 관련해 손실의 변화율을 계산하여 손실이 줄어드는 방향으로 파라미터 값을 바꿔나가는 최적화 알고리즘입니다.	$w = w - LearningRate \times LossRateOfChangeW$	수백만 개의 파라미터를 가진 대규모 신경망에도 쉽게 확장 적용할 수 있는 단순하고 강력한 개념입니다.	기울기(gradient)를 따라 최소값으로 수렴해가는 과정이며, 학습률 (learning rate) 조절이 학습의 성공을 결정짓는 중요한 요소입니다.
자동미분 (Autograd)	파이토치 텐서가 연산 그래프를 기억하여 미분을 자동으로 수행해주는 기능입니다.	<code>loss.backward()</code>	<code>requires_grad=True</code> 설정 시 연산 트리를 기록하며, <code>backward()</code> 호출 시 역방향으로 기울기를 계산합니다.	복잡한 신경망 모델에서 수작업으로 미분식을 작성해야 하는 수고를 덜어주며, 딥러닝 구현의 편의성을 극대화합니다.
옵티마이저 (Optimizer)	다양한 최적화 알고리즘(SGD, Adam 등)을 추상화하여 파라미터 업데이트를 대신 수행해주는 모듈입니다.	<code>optimizer.step()</code>	<code>zero_grad()</code> 로 기울기를 초기화하고 <code>step()</code> 으로 최적화 전략에 따라 파라미터 값을 조정합니다.	수동으로 파라미터를 업데이트하는 지루한 작업을 자동화하며, 학습 상황에 따라 적절한 알고리즘을 골라 쓸 수 있게 합니다.
과적합 (Overfitting)	모델이 훈련 데이터의 상세한 소음까지 학습하여 새로운 데이터(검증 세트)에서 성능이 떨어지는 현상입니다.	<pre>assert val_loss.requires_grad == False</pre>	훈련 손실은 낮으나 검증 손실이 높을 때 발생하며, 모델의 일반화 성능이 부족함을 의미합니다.	케플러가 데이터를 쪼개 일부만 사용하고 나머지는 검증을 위해 남겨둔 것처럼, 현대 머신러닝에서도 훈련/검증 데이터 분리는 필수적입니다.

코드를 수식으로 변환

손실 함수(loss_fn)를 L , 예측 모델(model)을 M , 그리고 미세한 변화량인 delta를 δ 라고 표현하면, 해당 코드는 아래와 같은 수식으로 나타낼 수 있음.

변화율 $\approx \frac{L(M(t_u, w+\delta, b), t_c) - L(M(t_u, w-\delta, b), t_c)}{2\delta}$

이 수식을 더 직관적으로 이해하기 위해, 다른 값들은 모두 고정되어 있고 오직 가중치 w 만 변할 때의 손실값을 함수 $f(w)$ 라고 축약하면 다음과 같이 아주 간단해짐.

기울기 $\approx \frac{f(w+\delta) - f(w-\delta)}{2\delta}$

이게 왜 기울기(Gradient)인가?

이 수식이 기울기인 이유는 우리가 중고등학교 수학에서 배운 '두 점 사이의 기울기 구하는 공식' 및 '미분의 정의'와 완전히 똑같기 때문임.

- x 변화량 분의 y 변화량: 기울기는 기본적으로 $\frac{y\text{의 변화량}}{x\text{의 변화량}}$ 으로 구함. 여기서 x 축은 파라미터인 w 이고, y

축은 손실(Loss)값임. w 를 기준으로 아주 조금 뒤쪽인 $w - \delta$ 에서 아주 조금 앞쪽인 $w + \delta$ 까지 이동했다고 가정해 보겠음. 이때 x 축의 전체 이동 거리(변화량)는 $(w + \delta) - (w - \delta)$ 이므로 2δ 가 됨. 이 식의 분모에 있는 `2.0 * delta`가 바로 이 x 의 변화량임. 분자는 당연히 그 두 지점에서의 손실값(y 값)의 차이를 의미함.

- **미분(순간 변화율)과의 관계:** 자료에 따르면, 두 점 사이의 거리를 의미하는 δ 를 극단적으로 0에 가깝게 줄이면 어떻게 될까? 이는 수학적으로 파라미터 w 에 대해 손실 함수를 '미분'하는 것과 정확히 일치함. 이 편미분 값들을 벡터에 넣은 것이 바로 딥러닝에서 말하는 **기울기(Gradient)**임. 즉, 코드로 작성한 저 수식은 0에 가까운 아주 작은 값(0.1)을 이용해 손실 함수 그래프 위의 특정 점(현재 w 위치)에서 접선의 기울기를 근사치로 계산한 것임.

요약하자면: 해당 코드는 기계의 w 라는 손잡이를 양쪽으로 아주 미세하게(δ) 돌려보면서, 그 사이클에서 에러(손실)가 얼마나 가파르게 올라가거나 내려가는지를 수치로 확인하는 과정임. 이 가파른 정도가 곧 **기울기**이며, 이 값을 알아야 손실이 줄어드는 방향(내리막길)으로 파라미터 값을 바꿔나갈 수 있음.